IBKR Paper Dry Run

Phase D one-day rehearsal against IB Gateway in **read-only** mode. No real orders are placed — the goal is to verify every piece of plumbing works before the 15-day paper week starts.

Before you start

You've already done `podman compose up -d`. Confirm these last few prereqs:

• IB Gateway is running, paper account, port 4002 (NOT 4001).
• Account ID starts with `DU`.
• `.env` has the IBKR settings (`IBKR_MODE=paper`, `IBKR_PORT=4002`, `IBKR_CLIENT_ID=42`, `IBKR_READONLY=true`).
• Source tree is clean: `git status -- PythonDataService references/qc-shadow` is empty.
• Activate the host venv once per shell:

```
cd PythonDataService
source .venv/Scripts/activate # Git Bash on Windows
# or: .venv\Scripts\Activate.ps1 (PowerShell)
cd ..
```

Set these shell variables once. The steps below reference them as `$ACCOUNT` and `$RUN_ID` so you don't paste raw angle-bracket placeholders that bash treats as redirects.

```
export ACCOUNT='DU1234567' # ← replace with your DU paper account
export PYTHONPATH=PythonDataService
# RUN_ID is set after Step 1; Steps 2 and 4 use it.
```

Step 1 — Initialize the dry-run ledger (HOST)

What: writes `run_ledger.json` with the run identity (strategy spec hash, QC audit copy hash, account ID, start-of-session UTC ms).

Why: the ledger is the canonical fingerprint for this run. The hashes in it appear in every reconciliation receipt, so a future operator can verify exactly what code and what spec produced the day's output. The init-ledger step refuses if your tree is dirty — that's how `code_sha` stays meaningful.

Single-line command for clean copy-paste — line wraps in the PDF are display only:

```
python -m app.engine.live.run init-ledger --repo-root . --clean-tree-scope PythonDataService
references/qc-shadow --strategy-spec-path
PythonDataService/app/engine/strategy/spec/fixtures/spy_ema_crossover.spec.json
--qc-audit-copy-path references/qc-shadow/SpyEmaCrossoverAlgorithm.py --qc-cloud-backtest-id
dry-run-no-cloud --account-id "$ACCOUNT" --start-date-ms $(date -u -d "today 09:30 EDT"
+%s000) --live-config-json '{"symbol":"SPY","force_flat_at":"15:55","client_id":42}'
--run-root PythonDataService/artifacts/live_runs
```

Expect: a single stdout line of the form `[INIT-LEDGER] wrote ... (run_id=<hex64>)`. Capture the run_id into a shell variable — Steps 2 and 4 reference it:

```
export RUN_ID='paste_the_64-char_hex_here'
```

Step 2 — Pre-flight gate (HOST)

What: runs the morning halt checks — clean tree, NTP offset, `run_ledger.json` intact, no leftover `halt.flag` from a prior session.

Why: this is the same gate that fires every morning during paper week. If any check fails, paper week is paused for the day. The dry run rehearses it so you know what "green" looks like before doing it for real.

```
python -m app.engine.live.run pre-flight --repo-root . --clean-tree-scope PythonDataService
references/qc-shadow --run-dir PythonDataService/artifacts/live_runs/$RUN_ID
```

Expect: every check prints `OK`, ending with `all checks passed; runner may proceed`. If `ntp_offset` fails because of a corporate firewall, add `--skip-ntp` for the dry run only (NOT for paper week — fix the firewall first).

Step 3 — Read-only run (CONTAINER)

What: connects to IB Gateway, subscribes to SPY 5-second TRADES bars, consolidates to 15-min, runs the strategy. `--readonly` short-circuits `place_order` so no broker orders go out.

Why: this is the only step that needs the container — the IBKR Gateway sidecar network lives there. Run it through a full session (or a synthetic replay window) to prove the bar stream, consolidator, decision logger, and writer pipeline all work end-to-end.

```
podman exec polygon-data-service python -m app.engine.live.run start --run-dir
/app/artifacts/live_runs/$RUN_ID --readonly
```

Expect during the run: IB Gateway shows one connected client (id=42); `decisions.parquet` grows by one row every 15 minutes; `executions.parquet` stays empty (correct — readonly). If the strategy emits an ENTER/EXIT signal, `decisions.parquet` records it but no fill comes back.

> **If a halt fires intra-day:** the runner stops, writes `halt.flag` or `poisoned.flag`, and exits non-zero. Inspect, decide if it's expected, then proceed.

Step 4 — End-of-session reconciliation (HOST)

What: compares the runner's `decisions.parquet` against a synthetic QC export, classifies every bar as `none` / `data` / `engine` divergence, and writes the day-0 Markdown receipt with a SHA-256 manifest of every artifact it summarizes.

Why: proves the daily reconcile pipeline works end-to-end. The committed Markdown is your dry-run deliverable — it's the same shape the operator will eyeball every paper day.

First, build a tiny synthetic QC indicators export:

```
export TODAY=$(date -u +%Y-%m-%d)
mkdir -p PythonDataService/artifacts/qc-dry-run/$TODAY
# Hand-craft indicators.csv from the runner's decisions.parquet
# (columns: bar_close_ms, ema5, ema10, rsi, signal). Worked example:
# docs/references/reconciliations/dry-run-2026-05-09/day-0.md
```

Then run reconcile:

```
python -m app.engine.live.reconcile --run-dir PythonDataService/artifacts/live_runs/$RUN_ID
--qc-dir PythonDataService/artifacts/qc-dry-run/$TODAY --docs-dir
docs/references/reconciliations/dry-run-$TODAY --run-label dry-run-$TODAY --day-n 0 --day-date
$TODAY
```

Expect: all four artifacts written (`day-0.md`, `.json`, `.parquet`, `.hashes.json`). Markdown shows zero **cross-engine** divergences.

> **Expected fill-class breach in readonly:** if the strategy emitted any signal in Step 3, the receipt will show `fill-class breach count=N` and write `halt.flag`. **This is normal in readonly mode** — readonly means no fills came back to match the ENTER/EXIT intent. The receipt is still valid. **Delete** `halt.flag` **before any subsequent pre-flight**, otherwise tomorrow's gate will refuse to proceed.

Step 5 — Regression test (HOST)

What: re-runs the live-engine test suite against the run-start commit.

Why: confirms nothing on master regressed since you started. Run from the host because `tests/` isn't mounted into the container.

```
cd PythonDataService
python -m pytest tests/engine/live/ -v
cd ..
```

Expect: ~165 passed, a few skipped (the QC-export-required test consumers wait for your QC Cloud Test 1/2 runs, which are a separate operator task).

You're done with Phase D when…

✓ Step 1 wrote a 64-char hex `run_id`.
✓ Step 2 emitted only `OK` lines.
✓ Step 3 ran a full session with no unexpected intra-day halts.
✓ Step 4 produced `day-0.md` (fill-class breach counted as expected if signals fired).
✓ Step 5 saw no regressions vs the prior baseline.
✓ No `halt.flag` or `poisoned.flag` left in the run dir.

If something goes wrong

Dry-tree halt: don't `git stash` to make it pass — that breaks `code_sha` identity. Commit or revert in scope and re-run from Step 1.
NTP halt: fix the network or pick a different `--ntp-server`. Don't `--skip-ntp` in paper week.
IB Gateway disconnects: the runner reconnects with a 60s timeout. On timeout it halts and writes a partial reconciliation. Resuming after disconnect requires a fresh `run_id`.
Reconcile sees engine-class divergence on dry-run synthetic inputs: that's a real bug in the classifier — file an issue and stop. Don't proceed to paper week.

After Phase D

Once Phase D is green, the remaining operator tasks are: (1) run QC Cloud Test 1 + Test 2 and commit the exports under `references/qc-shadow/backtests/` (this activates the skip-marked Test 1/2 consumers); (2) flip `IBKR_READONLY=false` in `.env`, re-run Step 1 to mint a new `run_id` for the live config delta, and start the 15-day paper week proper.